Buổi 2

Các mở rộng của C++ so với C (phần 1)

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Giới thiệu C++

1.1 Lịch sử C++ và chương trình C++ đầu tiên
1.2 Các từ khóa mới và cách ghi chú thích
1.3 Dòng nhập/xuất chuẩn: cin và cout
1.4 Định dạng dữ liệu xuất
1.5 Chuyển đổi kiểu và vị trí khai báo biến
1.6 Biến const
1.7 struct, union, enum và toán tử phạm vi ::
1.8 Kiểu bool và lớp string của thư viện chuẩn

2. Cấp phát bộ nhớ động: new và delete

2.1 Toán tử new và delete
2.2 Mảng động một chiều và hai chiều
2.3 Các lỗi thường gặp với new và delete
2.4 Ví dụ: mảng động tăng kích thước

3. Tổng kết và thực hành

3.1 Ví dụ tổng hợp: viết lại chương trình C sang C++
3.2 Bài thực hành buổi 2
3.3 Câu hỏi và bài tập

1.1 Lịch sử C++ và chương trình C++ đầu tiên

- **C++** do **Bjarne Stroustrup** (Bell Labs) phát triển từ 1979, ban đầu tên "C with Classes", đổi thành C++ năm 1983 (`++` là toán tử tăng của C ⇒ "C được nâng cấp").
- C++ là **siêu tập** (gần như) của C: hầu hết chương trình C biên dịch được bằng trình biên dịch C++.
- C++ bổ sung: lớp và đối tượng, kế thừa, đa hình, đa năng hóa hàm/toán tử, tham chiếu, template, xử lý ngoại lệ, thư viện chuẩn STL.
- Các chuẩn: C++98, C++03, C++11, C++14, C++17, C++20…
- Tập tin nguồn C++ có phần mở rộng **.cpp**; biên dịch: `g++ ten.cpp -o ten`.

Ví dụ 2.1 – So sánh chương trình C và C++

```c
/* C */
#include <stdio.h>
int main()
{  int n;
   printf("Nhap n: ");
   scanf("%d", &n);
   printf("Binh phuong = %d\n", n * n);
   return 0;
}
```

```cpp
// C++
#include <iostream>
using namespace std;
int main()
{  int n;
   cout << "Nhap n: ";
   cin >> n;
   cout << "Binh phuong = " << n * n << endl;
   return 0;
}
```

- `using namespace std;` cho phép dùng tên `cout`, `cin`, `endl` thay vì `std::cout`, …

1.2 Các từ khóa mới và cách ghi chú thích

- **Các từ khóa mới của C++** (không dùng làm tên biến, tên hàm):

Nhóm	Từ khóa
Lớp, đối tượng	class, public, private, protected, this, friend
Bộ nhớ	new, delete
Kế thừa, đa hình	virtual, operator
Hàm	inline, template
Ngoại lệ	try, catch, throw
Kiểu, khác	bool, true, false, namespace, using, const_cast…

Ví dụ 2.2 – Lỗi khi dịch mã C cũ bằng C++

`int new = 5; char *class = "A";` hợp lệ trong C nhưng **lỗi** trong C++ vì `new`, `class` là từ khóa ⇒ phải đổi tên biến.

- **Chú thích:**
  - Kiểu C: `/* … */` – có thể nhiều dòng, **không** lồng nhau được.
  - Kiểu C++: `// …` – chú thích **đến hết dòng**.

```
int Dem = 0;      // đếm số sinh viên đạt
/* Hàm tính điểm trung bình
   có trọng số theo số tín chỉ */
float TinhDTB(float d[], int tc[], int n);
```

- **Quy ước:** dùng `//` cho chú thích ngắn cuối dòng; dùng `/* */` để tạm thời vô hiệu hóa một khối mã (nhưng cẩn thận khi khối đó đã chứa `/* */`).

Lỗi thường gặp

```
x = a /* b */ / c;     // hợp lệ: x = a / c
int y = 10; //* chú thích */ 2;  // // chiếm ưu tiên ⇒ y =
10
```

1.3 Dòng nhập/xuất chuẩn: cin và cout

- Thư viện `<iostream>`: **cout** – dòng xuất chuẩn (màn hình), dùng **toán tử chèn** `<<`; **cin** – dòng nhập chuẩn (bàn phím), dùng **toán tử trích** `>>`; **cerr** – xuất thông báo lỗi.

```
int a;  float b;  char c;  char s[30];
cout << "Nhap a, b, c: ";
cin >> a >> b >> c;            // nhập nhiều giá trị liên
tiếp
cout << "a = " << a << ", b = " << b
     << ", c = " << c << endl; // endl: xuống dòng
cin >> s;                      // đọc một TỪ (dừng ở khoảng
trắng)
cin.ignore();                  // bỏ ký tự '\n' còn lại
cin.getline(s, 30);            // đọc cả DÒNG có khoảng trắng
```

- **Ưu điểm so với printf/scanf:**
  - **Không cần** chuỗi định dạng `%d`, `%f` – trình biên dịch tự nhận kiểu ⇒ tránh lỗi sai định dạng.
  - **Không cần** dấu `&` khi nhập.
  - Mở rộng được cho kiểu do người dùng định nghĩa (đa năng hóa `<<`, `>>` – buổi 8).

Ví dụ 2.3 – Lỗi kinh điển của C mà C++ tránh được

```
/* C */  float x;  scanf("%d", &x);   /* sai định dạng ⇒ x
rác */
         int n;    scanf("%d", n);    /* quên & ⇒ lỗi bộ
nhớ  */
// C++   float x;  cin >> x;          // đúng kiểu tự động
```

Ví dụ 2.4 – Nhập họ tên có khoảng trắng

Nhập `Nguyen Van An`:

- `cin >> s;` ⇒ s = **"Nguyen"** (phần còn lại vẫn nằm trong bộ đệm).
- `cin.getline(s, 30);` ⇒ s = **"Nguyen Van An"**.
- Nếu trước đó vừa `cin >> tuoi;` thì phải `cin.ignore()` để bỏ ký tự xuống dòng, nếu không getline đọc được chuỗi **rỗng**.

1.4 Định dạng dữ liệu xuất

- Thư viện `<iomanip>` cung cấp các **bộ điều khiển** (manipulator) định dạng:

Bộ điều khiển	Tác dụng
`setw(n)`	Độ rộng vùng xuất n ký tự (chỉ áp dụng cho **giá trị kế tiếp**)
`left`, `right`	Canh trái / phải trong vùng setw
`setprecision(n)`	Số chữ số (dùng với `fixed`: số chữ số sau dấu chấm)
`fixed`	Xuất số thực dạng dấu chấm tĩnh
`setfill(c)`	Ký tự lấp đầy vùng trống
`hex`, `oct`, `dec`	Hệ 16, 8, 10
`boolalpha`	Xuất bool dạng true/false

Ví dụ 2.5 – Chương trình và kết quả

```
cout << setw(6) << 42 << "|" << endl;
cout << left << setw(6) << 42 << "|" << right << endl;
cout << fixed << setprecision(2) << 3.14159 << endl;
cout << setfill('0') << setw(5) << 7 << setfill(' ') <<
endl;
cout << hex << 255 << " " << oct << 8 << dec << " " << 10
<< endl;
cout << boolalpha << (3 > 2) << endl;
```

```
    42|
42    |
3.14
00007
ff 10 10
true
```

Ví dụ 2.6 – In bảng điểm thẳng cột

```
cout << left << setw(10) << "MaSV" << setw(20) << "Ho ten" << right << setw(6) << "DTB" << endl;
cout << fixed << setprecision(1);
for (int i = 0; i < n; i++)
   cout << left << setw(10) << ma[i] << setw(20) << ten[i] << right << setw(6) << dtb[i] << endl;
```

- Lưu ý: `fixed`, `setprecision`, `left`, `hex` có tác dụng **lâu dài** (đến khi đổi lại); riêng `setw` chỉ áp dụng cho **một** lần xuất.

1.5 Chuyển đổi kiểu và vị trí khai báo biến

- **Chuyển đổi kiểu tường minh:** ngoài cách viết của C `(kiểu) biểu_thức`, C++ cho phép viết **như gọi hàm** `kiểu(biểu_thức)`:

```
double d = 7.9;
int i = int(d);            // C++: kiểu hàm ⇒ 7
int j = (int)d;            // kiểu C vẫn dùng được
cout << 7/2 << " " << double(7)/2;   // 3  3.5
```

- C++ còn có các toán tử ép kiểu an toàn hơn: `static_cast<int>(d)`, `const_cast`, `dynamic_cast` (buổi 10), `reinterpret_cast`.

Ví dụ 2.7 – Tính trung bình cộng

```
int tong = 17, n = 4;
cout << tong / n;              // 4   (chia nguyên!)
cout << double(tong) / n;      // 4.25
cout << double(tong / n);      // 4   (ép kiểu SAU khi đã
chia)
```

- **Vị trí khai báo biến:** C (C89) bắt buộc khai báo ở **đầu khối**; C++ cho phép khai báo **ở bất kỳ đâu**, ngay **trước khi dùng** – kể cả trong vòng for.

```
for (int k = 0; k < 3; k++)    // k chỉ tồn tại trong vòng
for
   cout << k << ' ';           // 0 1 2
// cout << k;                  // LỖI: k không còn phạm vi
int tong = 0;                  // khai báo gần nơi sử dụng
for (int i = 0; i < n; i++) tong += a[i];
```

- **Lợi ích:** biến có phạm vi hẹp ⇒ ít nhầm lẫn; khởi tạo ngay khi khai báo ⇒ tránh biến chưa có giá trị.
- **Lưu ý:** không được nhảy (goto, case của switch) qua lệnh khai báo có khởi tạo.

1.6 Biến const

- **const** khai báo **hằng có kiểu**: phải khởi tạo khi khai báo, sau đó **không thể thay đổi**.

```
const int MAX = 100;           // thay cho #define MAX 100
const double PI = 3.14159;
int a[MAX];                    // dùng làm kích thước mảng
```

Tiêu chí	`#define MAX 100`	`const int MAX = 100;`
Xử lý bởi	Tiền xử lý (thay thế văn bản)	Trình biên dịch
Kiểm tra kiểu	Không	**Có**
Phạm vi	Từ chỗ define đến hết tập tin	Theo khối như biến
Gỡ lỗi	Tên MAX biến mất sau tiền xử lý	Thấy tên MAX

- **const với con trỏ** – đọc từ phải sang trái:

Khai báo	Ý nghĩa
`const int *p`	Con trỏ tới **hằng**: không sửa `*p`, được đổi p
`int * const q = &a`	**Hằng con trỏ**: không đổi q, được sửa `*q`
`const int * const r = &a`	Không đổi cả r lẫn `*r`

Ví dụ 2.8 – Thông báo lỗi của g++

```
const int *p = &a;  int * const q = &a;
p = &b;   *q = 7;          // hợp lệ
*p = 3;   // error: assignment of read-only location '* p'
q = &b;   // error: assignment of read-only variable 'q'
N = 6;    // error: assignment of read-only variable 'N'
```

- **Ứng dụng quan trọng:** tham số `const char *s` cam kết hàm **không sửa** chuỗi truyền vào; về sau: hàm thành viên const, đối tượng hằng (buổi 6).

1.7 struct, union, enum và toán tử phạm vi ::

- **Tên struct, union, enum là tên kiểu** – không cần `typedef` hay từ khóa `struct` khi khai báo biến:

```
struct Diem { int x, y; };
Diem p = {3, 4};            // C phải viết: struct Diem p
enum Mau { DO, XANH, VANG };
Mau m = XANH;               // m = 1
```

- **struct trong C++ có thể chứa hàm** – bước đầu tiến tới lớp (buổi 4):

```
struct Diem {
   int x, y;
   void In() { cout << "(" << x << ", " << y << ")"; }
};
p.In();                     // (3, 4)
```

- **Union vô danh:** các thành phần dùng chung vùng nhớ, truy cập trực tiếp bằng tên.

```
union { int i; char c[4]; } u;  u.i = 0x41424344;
cout << u.c[0];             // 'D' trên máy little-endian
```

- **Toán tử định phạm vi `::`**
  - `::ten` – truy cập **biến toàn cục** bị che bởi biến cục bộ cùng tên.
  - `Lop::ten` – thành viên của lớp (định nghĩa hàm ngoài lớp – buổi 4).
  - `std::cout` – tên trong không gian tên std.

Ví dụ 2.9 – Chương trình và kết quả

```
int x = 10;                 // toàn cục
int main()
{  int x = 20;              // cục bộ che toàn cục
   cout << "x cuc bo = " << x
        << ", x toan cuc = " << ::x << endl;
   ::x = 99;                // sửa biến toàn cục
}
```

```
x cuc bo = 20, x toan cuc = 10
```

1.8 Kiểu bool và lớp string của thư viện chuẩn

- **Kiểu bool:** giá trị `true` / `false` (C chỉ dùng int 0/1; C99 mới có `_Bool`).

```
bool ok = 5 > 3;
cout << ok << " " << sizeof(bool);       // 1 1
cout << boolalpha << ok;                 // true
bool LaSoNguyenTo(int n)
{  if (n < 2) return false;
   for (int i = 2; i * i <= n; i++)
      if (n % i == 0) return false;
   return true;
}
```

- Chuyển đổi: số khác 0 ⇒ `true`, 0 ⇒ `false`; `true` ⇒ 1, `false` ⇒ 0.

- **Lớp string** (`#include <string>`) – chuỗi ký tự **tự quản lý bộ nhớ**, dùng như kiểu cơ sở:

```
string ho = "Nguyen", ten = "An";
string hoTen = ho + " Van " + ten;       // nối bằng +
cout << hoTen << " | " << hoTen.length()
     << " | " << (ho < ten)               // so sánh bằng <
     << " | " << hoTen.substr(7, 3);      // chuỗi con
getline(cin, hoTen);                      // nhập cả dòng
```

```
Nguyen Van An | 13 | 0 | Van
```

Chuỗi C (`char s[30]`)	`string` (C++)
`strcpy(s, t)`	`s = t`
`strcat(s, t)`	`s += t`
`strcmp(s, t) == 0`	`s == t`
`strlen(s)`	`s.length()`
Tràn nếu chuỗi dài hơn 29 ký tự	Tự cấp phát lại

- `string` là một **lớp** – ví dụ đầu tiên về sức mạnh của OOP: đóng gói con trỏ, cấp phát động, đa năng hóa toán tử `+`, `==`, `<<` (buổi 8 sẽ tự xây dựng lớp String).

2.1 Toán tử new và delete

- C dùng hàm `malloc/calloc/free` (thư viện `<stdlib.h>`); C++ dùng **toán tử** `new` và `delete` – gọn hơn, an toàn kiểu, và tự gọi **constructor/destructor** cho đối tượng (buổi 5).

Cấp phát	Giải phóng	Ý nghĩa
`p = new T;`	`delete p;`	Một biến kiểu T
`p = new T(giá_trị);`	`delete p;`	Một biến, có khởi tạo
`p = new T[n];`	`delete[] p;`	Mảng n phần tử (n có thể là biến)

```
int *q = new int(99);
cout << *q << endl;        // 99
delete q;
```

- Nếu không đủ bộ nhớ, `new` ném ngoại lệ `bad_alloc` (C++ hiện đại); dạng `new (nothrow) T[n]` trả về **NULL** như giáo trình mô tả.

Tiêu chí	malloc / free (C)	new / delete (C++)
Bản chất	Hàm thư viện	Toán tử của ngôn ngữ
Kích thước	Tự tính `n*sizeof(int)`	Tự động theo kiểu
Kiểu trả về	`void*` – phải ép kiểu	Đúng kiểu `int*`
Khởi tạo đối tượng	Không gọi constructor	**Gọi constructor**
Giải phóng đối tượng	Không gọi destructor	**Gọi destructor**

Ví dụ 2.10 – Cùng một việc

```
int *a = (int*)malloc(n * sizeof(int));  free(a);   // C
int *b = new int[n];                     delete[] b; // C++
```

- **Quy tắc:** cấp phát bằng gì thì giải phóng bằng cái đó; **không** trộn malloc với delete hay new với free.

Ví dụ 2.11 – Mảng động 1 chiều với kích thước nhập lúc chạy

```
int n;
cout << "So phan tu: ";  cin >> n;
int *a = new int[n];              // kích thước là biến
for (int k = 0; k < n; k++) a[k] = k * k;
for (int k = 0; k < n; k++) cout << a[k] << ' ';
delete[] a;                       // [] vì là mảng
a = NULL;                         // tránh con trỏ treo
```

- n = 4 ⇒ in ra: `0 1 4 9`
- Truy cập phần tử như mảng thường: `a[k]` hoặc `*(a + k)`.

Ví dụ 2.12 – Ma trận động m × n

```
int **mt = new int*[m];           // mảng m con trỏ dòng
for (int r = 0; r < m; r++)
   mt[r] = new int[n];            // mỗi dòng n phần tử
for (int r = 0; r < m; r++)
   for (int c = 0; c < n; c++)
      mt[r][c] = r * n + c;
cout << mt[1][2];                 // m = 2, n = 3 ⇒ 5
for (int r = 0; r < m; r++)       // giải phóng NGƯỢC thứ tự
   delete[] mt[r];
delete[] mt;
```

```
mt ─► [●]──► [0][1][2]
      [●]──► [3][4][5]
```

2.3 Các lỗi thường gặp với new và delete

Lỗi	Ví dụ	Hậu quả
Rò rỉ bộ nhớ (memory leak)	`p = new int[100]; p = new int[200];` (mất địa chỉ vùng đầu)	Bộ nhớ không thu hồi được, chương trình chạy lâu sẽ cạn RAM
Dùng `delete` cho mảng	`int *a = new int[10]; delete a;`	Hành vi không xác định (với mảng đối tượng: chỉ gọi 1 destructor)
Giải phóng **hai lần**	`delete p; delete p;`	Hỏng vùng quản lý heap, crash
Dùng sau khi giải phóng (con trỏ treo)	`delete p; *p = 5;`	Ghi đè vùng nhớ của dữ liệu khác
Giải phóng biến không cấp phát động	`int x; delete &x;`	Crash
Truy cập ngoài mảng	`a = new int[n]; a[n] = 0;`	Hỏng heap, lỗi khó tìm

Ví dụ 2.13 – Hàm tạo mảng động: đúng và sai

```
// SAI: con trỏ a truyền theo giá trị
void TaoMang(int *a, int n) { a = new int[n]; }
// ĐÚNG cách 1: trả về con trỏ
int *TaoMang(int n) { return new int[n]; }
// ĐÚNG cách 2: tham chiếu tới con trỏ (buổi 3)
void TaoMang(int *&a, int n) { a = new int[n]; }
```

- Bản SAI: con trỏ ở hàm gọi **không** nhận được vùng nhớ mới, đồng thời vùng nhớ bị **rò rỉ**.

- **Nguyên tắc an toàn:**
  - Mỗi `new` có đúng **một** `delete` tương ứng; `new[]` đi với `delete[]`.
  - Sau `delete p;` gán `p = NULL;` (`delete NULL` là an toàn, không làm gì).
  - Biên dịch với `-fsanitize=address` để phát hiện lỗi bộ nhớ.
  - C++ hiện đại: dùng `vector`, `string`, con trỏ thông minh (`unique_ptr`) để tránh tự quản lý bộ nhớ – nhưng cần hiểu new/delete trước để nắm cơ chế constructor/destructor, sao chép sâu (buổi 5, 8).

2.4 Ví dụ: mảng động tăng kích thước

- Mảng cấp phát bằng `new` **không thể** "nới rộng tại chỗ" (C++ không có realloc tương ứng) ⇒ cấp phát vùng mới, chép dữ liệu, giải phóng vùng cũ.

```
int *ThemPhanTu(int *a, int &n, int x)
{  int *b = new int[n + 1];     // 1. vùng mới
   for (int i = 0; i < n; i++)
      b[i] = a[i];               // 2. chép dữ liệu cũ
   b[n] = x;                     // 3. thêm phần tử mới
   delete[] a;                   // 4. giải phóng vùng cũ
   n++;                          // 5. cập nhật kích thước
   return b;
}
int main()
{  int n = 0;  int *a = NULL;
   a = ThemPhanTu(a, n, 5);
   a = ThemPhanTu(a, n, 8);
   a = ThemPhanTu(a, n, 2);
   for (int i = 0; i < n; i++) cout << a[i] << ' ';
   cout << "(n = " << n << ")" << endl;
   delete[] a;
}
```

```
5 8 2 (n = 3)
```

- **Phân tích:**
  - Lần gọi đầu a = NULL: `delete[] NULL` an toàn.
  - Tham số `int &n` (tham chiếu – buổi 3) để hàm **cập nhật** được n của hàm gọi.
  - Phải **gán lại** `a = ThemPhanTu(…)`; nếu quên, a vẫn trỏ vùng **đã giải phóng** (con trỏ treo).
  - Mỗi lần thêm phải chép n phần tử ⇒ thêm N phần tử tốn $O(N^2)$. Cải tiến: cấp phát **gấp đôi** khi đầy (cách `vector` hoạt động).

Ví dụ 2.14 – Kiểm tra bằng AddressSanitizer

```
g++ -fsanitize=address -g bai.cpp -o bai
```

- Nếu bỏ dòng `delete[] a;` cuối main ⇒ khi kết thúc in: `ERROR: LeakSanitizer: detected memory leaks … 12 byte(s) leaked.`
- Nếu dùng `delete a;` thay `delete[] a;` ⇒ báo `alloc-dealloc-mismatch`.

3.1 Ví dụ tổng hợp: viết lại chương trình C sang C++

```c
/* C: nhập n số, in số lớn nhất */
#include <stdio.h>
#include <stdlib.h>
#define MAX_N 1000
int main()
{  int n, i, max, *a;
   printf("n = "); scanf("%d", &n);
   if (n > MAX_N) n = MAX_N;
   a = (int*)malloc(n * sizeof(int));
   for (i = 0; i < n; i++)
   {  printf("a[%d] = ", i);
      scanf("%d", &a[i]);
   }
   max = a[0];
   for (i = 1; i < n; i++)
      if (a[i] > max) max = a[i];
   printf("Max = %d\n", max);
   free(a);
   return 0;
}
```

```cpp
// C++
#include <iostream>
using namespace std;
const int MAX_N = 1000;          // const thay #define
int main()
{  int n;
   cout << "n = ";  cin >> n;    // cin/cout
   if (n > MAX_N) n = MAX_N;
   int *a = new int[n];          // new thay malloc
   for (int i = 0; i < n; i++)   // khai báo i trong for
   {  cout << "a[" << i << "] = ";
      cin >> a[i];
   }
   int max = a[0];               // khai báo tại chỗ dùng
   for (int i = 1; i < n; i++)
      if (a[i] > max) max = a[i];
   cout << "Max = " << max << endl;
   delete[] a;                   // delete[] thay free
   return 0;
}
```

- **5 thay đổi:** iostream, const, new/delete[], khai báo biến tại chỗ, cin/cout không cần định dạng và &.

3.2 Bài thực hành buổi 2

Bài 1 – Viết lại chương trình C sang C++

Cho chương trình C quản lý điểm (nhập n sinh viên gồm mã, họ tên có khoảng trắng, điểm; in bảng điểm; tìm SV điểm cao nhất). Viết lại bằng C++: dùng `cin/cout`, `getline`, `setw`, `setprecision`, `const`, khai báo biến tại chỗ, `struct SinhVien` không cần typedef.

Bài 2 – Cấp phát mảng động bằng new/delete

- a) Nhập n, cấp phát mảng n số thực; nhập, tính trung bình, đếm số phần tử lớn hơn trung bình; giải phóng.
- b) Viết hàm `int *ThemPhanTu(int *a, int &n, int x)` tạo mảng mới n + 1 phần tử, chép dữ liệu cũ, thêm x vào cuối, giải phóng mảng cũ, trả về mảng mới.
- c) Cấp phát ma trận động m × n; nhập; in ma trận chuyển vị (cũng cấp phát động); giải phóng đúng thứ tự.
- d) Biên dịch với `g++ -fsanitize=address` và cố tình bỏ một lệnh delete[] để quan sát thông báo rò rỉ bộ nhớ.

Tiêu chí chấm

Chương trình không dùng `printf/scanf/malloc/free`; mọi `new` đều có `delete` tương ứng; bảng in thẳng cột; xử lý nhập họ tên có khoảng trắng đúng.

3.3 Câu hỏi và bài tập

Câu hỏi

- 1) Nêu 3 ưu điểm của `cin/cout` so với `scanf/printf`.
- 2) So sánh `const` và `#define`. Phân biệt `const int *p` và `int * const p`.
- 3) So sánh `new/delete` với `malloc/free`. Khi nào dùng `delete[]`?
- 4) Toán tử `::` dùng trong những trường hợp nào?

Bài tập dự đoán kết quả (có đáp số)

```
int x = 5;
int main()
{  int x = 2;
   { int x = 7;  cout << x << " " << ::x << endl; }
   cout << x << " " << double(9)/2 << " " << 9/2 << endl;
   cout << setw(5) << setfill('*') << 42 << endl;
   const int *p = &x;  p = &::x;  cout << *p << endl;
}
```

→ **Đáp số:** `7 5` / `2 4.5 4` / `***42` / `5`.

Bài tập lập trình

- 1) Viết chương trình nhập một dòng văn bản, đếm số từ (dùng `cin.getline`).
- 2) Viết chương trình cấp phát động mảng các `struct PhanSo {int tu, mau;}`, nhập n phân số, in phân số lớn nhất.